Applied NLP

DATA 461/641

Lecture 5 — Text Representation

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, ISBN 9781617294631

Skip-gram vs. CBOW: When to use which approach

- Mikolov highlighted that the skip-gram approach works well with small corpora and rare terms. With the skip-gram approach, you will have more examples due to the network structure.

- The continuous bag-of-words approach shows higher accuracies for frequent words and is much faster to train.

Computational tricks of Word2Vec

1. Frequent bigrams

- Some words often occur in combination with other words. For example, "Elvis" is often followed by "Presley". Therefore form bigrams. Since the word "Elvis" would occur with "Presley" with a high probability, we don't really gain much value from this prediction.

- A solution to the above limitation is to identify bigrams and trigrams that should be considered single terms using the following scoring function.

$$score(w_i, w_j) = \frac{count(w_i, w_j) - \delta}{count(w_i)count(w_j)} \tag{1}$$

2. Subsampling frequent tokens

- Common words like "the" or "a" often do not carry significant information. And the co-occurrence of the word "the" with a broad variety of other nouns in the corpus might create less meaningful connections between words, muddying the Word2vec representation with this false semantic similarity training.

- To reduce the emphasis on frequent words like stop words, words are sampled during training in inverse proportion to their frequency. The effect of this is similar to the IDF effect on TF-IDF vectors. Frequent words are given less influence over the vector than the rarer words. Mikolov used the following equation to determine the probability of sampling a given word. This probability determines whether or not a particular word is included in a particular skip-gram during training

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_t)}} \tag{2}$$

- In the preceding equations, $f(w_t)$ represents the frequency of a word across the corpus, and t represents a frequency threshold above which you want to apply the sub-sampling probability. The threshold depends on your corpus size, average document length, and the variety of words used in those documents. Values between 10^{-5} and 10^{-6} are often found in the literature.

3. Negative sampling

- If a single training example with a pair of words is presented to the network, it will cause all weights for the network to be updated. This changes the values of all the vectors for all the words in your vocabulary.

- But if your vocabulary contains thousands or millions of words, updating all the weights for the large one-hot vector is inefficient. To speed up the training of word vector models, Mikolov used negative sampling.

- Instead of updating all word weights that weren't included in the word window, Mikolov suggested sampling just a few negative samples (in the output vector) to update their weights. Instead of updating all weights, you pick n negative example word pairs (words that do not match your target output for that example) and update the weights that contributed to their specific output. That way, the computation can be reduced dramatically and the performance of the trained network does not decrease significantly.

- If you train your word model with a small corpus, you might want to use a negative sampling rate of 5 to 20 samples. For larger corpora and vocabularies, you can reduce the negative sample rate to as low as two to five samples.